

Type-safe Bidirectional Channels in Idris 2

Guillaume Allais¹

University of Strathclyde, Glasgow, Scotland, United Kingdom
guillaume.allais@strath.ac.uk

Introduction

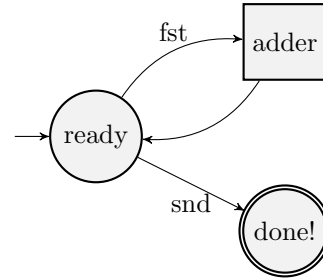
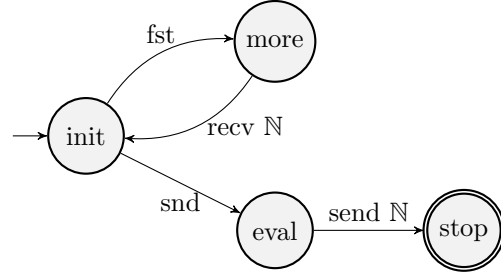
Session types are a channel typing discipline ensuring that the communication patterns of concurrent programs abide by a shared protocol. A meta-theoretical analysis of the typing discipline can then ensure that the communicating processes will have good properties e.g. deadlock-freedom. We show how to use a linear dependently typed programming language to define a direct and type-safe embedding of expressive binary session types.

Expressive Session Types

Our session types are closed under typed sends ($!A. \cdot$), typed receives ($?A. \cdot$), branch offerings ($\cdot \& \cdot$), branch selections ($\cdot \oplus \cdot$), and smallest ($\mu x. \cdot$) and largest ($\nu x. \cdot$) fixpoints.

The session $\nu x. ((?N. x) \& (!N. \text{end}))$ types a server for the “adder” protocol represented as a finite state machine on the right hand side. The server accepts an arbitrarily long sequence of natural numbers (sent by a client repeatedly selecting the *more* = $(?N. x)$ branch) before sending back their sum as a single number (when the *eval* = $(!N. \text{end})$ branch is ultimately selected) and terminating. The encoding of the server’s type uses a largest fixpoint because the client is the one responsible for ensuring the process terminates by eventually selecting the *eval* branch.

We can use nested fixpoint to describe more complex protocols. For instance, $\nu o. ((\nu x. ((?N. x) \& (!N. o))) \& \text{end})$ informally corresponds to the finite state machine shown on the right hand side. It types a server repeatedly offering to handle the adder protocol described above (we replaced *end* with *o* to allow for the repetition) before stopping when the client is done.

$$\begin{aligned} p, q, \dots ::= & x \mid \mu x. p \mid \nu x. p \\ & \mid !A. p \mid ?A. p \mid \text{end} \\ & \mid p \& q \mid p \oplus q \end{aligned}$$


Type-safe Implementation

Pairs of Untyped Channels We represent a bidirectional channel parametrised by a session type as a pair of untyped unidirectional channels. The key idea behind this safe implementation is to collect all the types used in a protocol and to manufacture a big sum type in which we

can inject all of the values that need to be exchanged. The definition of this sum type is a type safe realisation of Kiselyov and Sabry’s open union type [5] using the encoding of scope-safe De Bruijn indices [4] introduced by Brady in Idris 2’s core [3]. This gives us an encoding that has constant time injections and (partial) projections in and out of the big sum type.

Dealing with Changing Sessions The session type, by design, changes after each communication but the big sum type used when communicating across the untyped channels needs to stay the same. Our solution is to record an offset remembering where we currently are in the protocol and thus allowing us to keep injecting values to be sent into the initial shared sum type. Our offsets are proven correct using a gadget that can be understood as a cut down version of McBride’s one hole contexts [8]. Instead of recording the full path followed by our programs in the finite state machine describing the protocol, we merely record a de-looped version.

Code Samples We include code snippets showing the types of `offer` and `select` the dual primitives allowing branching. Both are linear in their channel input; both involve a choice in the form of a `Bool` value: `offer` gets it from the other thread while `select` requires the caller to pass it. In both cases, an appropriately stepped channel is return to continue communicating.

```
offer : LinearIO io => {s1, s2 : _} ->
  (1 ch : Channel (Offer s1 s2) e) ->
  L1 io (Res Bool $ \ b => ifThenElse b (Channel s1 e) (Channel s2 e))
```

```
select : LinearIO io => {s1, s2 : _} ->
  (1 ch : Channel (Select s1 s2) e) ->
  (b : Bool) -> L1 io (ifThenElse b (Channel s1 e) (Channel s2 e))
```

Limitations and Future Work

Encoding Uniqueness via Linear Types Unlike Brady’s prior work in Idris 1 [2], we are using Idris 2 which does not currently have uniqueness types. We are forced to use the linearity granted to us by Quantitative Type Theory (QTT) [9, 1] to emulate uniqueness via a library-wide invariant. This adds a degree of noise and trust to the implementation. We would ideally want a system combining both linearity and uniqueness to benefit from enhanced expressivity and efficiency as described by Marshall, Vollmer, and Orchard [7].

Small Runtime Overhead In the current version of our library, we compute a handful of key offset values by induction over the protocol. Correspondingly the protocol is not marked as erased anymore and, if compilation does not specialise the relevant combinators aggressively, then we may very well be evaluating these relatively small computations at runtime. As far as we understand, directly adding typed staging to Idris 2 through the use of a two-level type theory à la Kovács [6] would not be enough to solve our issue: for typing purposes, the protocol does need to persist through staging. But it should be possible to move it from QTT’s unrestricted to its erased modality.

Ergonomics We found writing the types of inner loops of programs with non-trivial communication patterns to be tedious as they expose quite a lot of the powerful but noisy encoding of syntaxes with binding we use. Figuring out a more user-friendly surface syntax for our protocols is left as future work.

References

- [1] Robert Atkey. Syntax and semantics of quantitative type theory. In Anuj Dawar and Erich Grädel, editors, *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018*, pages 56–65. ACM, 2018.
- [2] Edwin C. Brady. Type-driven development of concurrent communicating systems. *Comput. Sci.*, 18(3), 2017.
- [3] Edwin C. Brady. Idris 2: Quantitative type theory in practice. In Anders Möller and Manu Sridharan, editors, *35th European Conference on Object-Oriented Programming, ECOOP 2021, July 11-17, 2021, Aarhus, Denmark (Virtual Conference)*, volume 194 of *LIPICs*, pages 9:1–9:26. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.
- [4] Nicolaas Govert de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392, 1972.
- [5] Oleg Kiselyov, Amr Sabry, and Cameron Swords. Extensible effects: an alternative to monad transformers. In Chung-chieh Shan, editor, *Proceedings of the 2013 ACM SIGPLAN Symposium on Haskell, Boston, MA, USA, September 23-24, 2013*, pages 59–70. ACM, 2013.
- [6] András Kovács. Staged compilation with two-level type theory. *Proc. ACM Program. Lang.*, 6(ICFP):540–569, 2022.
- [7] Danielle Marshall, Michael Vollmer, and Dominic Orchard. Linearity and uniqueness: An entente cordiale. In Ilya Sergey, editor, *Programming Languages and Systems - 31st European Symposium on Programming, ESOP 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings*, volume 13240 of *Lecture Notes in Computer Science*, pages 346–375. Springer, 2022.
- [8] Conor McBride. Clowns to the left of me, jokers to the right (pearl): dissecting data structures. In George C. Necula and Philip Wadler, editors, *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008*, pages 287–295. ACM, 2008.
- [9] Conor McBride. I got plenty o’ nuttin’. In Sam Lindley, Conor McBride, Philip W. Trinder, and Donald Sannella, editors, *A List of Successes That Can Change the World - Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday*, volume 9600 of *Lecture Notes in Computer Science*, pages 207–233. Springer, 2016.